

Backorder Risk Capstone Showcase Reference

Predictive Supply Chain Analytics for Backorder Prevention and Inventory Optimization

Addy Cruz | DATA-470 Capstone | Westminster University
Showcase reference sheet

Quick Navigation			
PROJECT CORE		DEFENSE	TALK TRACK
30-Second Story		Model Definitions	Future Plans
Headline Result		Edge Cases	Glossary
Flow To Explain		Likely Questions	By Audience

30-Second Story	
Problem: backorders delay customers; overstock ties up capital. I used SAP order-line data to predict backorder risk early enough for planners to act. Final scope: v2 order-time binary classification: <i>backorder vs no backorder</i>. Why it matters: even when model deployment is blocked, the pipeline gives an evidence-based way to audit risk, data maturity, and operational readiness.	
31,177	3.38%
order lines	positive rate
6,521	58
test rows	test positives

Headline Result	
Selected champion OOF-calibrated stack, chosen by inner temporal validation. Deployable pick Logistic Regression, because it was strongest and simplest on outer temporal holdout. • LR temporal: ROC-AUC 0.910, PR-AUC 0.358, precision 0.453, recall 0.586, F1 0.511 at threshold 0.309. • Stack temporal: ROC-AUC 0.936, PR-AUC 0.326, precision 0.410, recall 0.431, F1 0.420. • Both pass model gates: precision ≥ 0.15 and recall ≥ 0.35.	

Flow To Explain

- **Raw SAP CSVs** -> cleaned sales, delivery, billing, inventory, purchasing, master-data tables.
- **Master fulfillment table** -> BRD metrics + modeling table.
- **v2 order-time features** only use facts available when the order is placed.
- **Temporal split:** train on older orders, test on later orders.
- **Model protocol:** choose with inner temporal validation; report final outer temporal holdout once.

Plain-English Temporal Split

Randomly mixing old and new orders is like studying with some of the test answers accidentally included. It can make the model look smarter than it really is because it has seen patterns from the same time period it is being tested on. I used older orders to predict newer orders because that matches the real question: can past history help us predict future backorders?

Leakage Guardrail

Excluded downstream snapshot/outcome-like columns: outstanding quantity, saleable inventory, delivery status, billing status, backorder units/amount, shipment date, etc.

Model Definitions

- **Logistic Regression:** linear baseline; interpretable; strong when signal separates mostly by additive feature weights.
- **Random Forest:** many decision trees bagged together; good at non-linear interactions, can over-warn on rare events.
- **XGBoost / LightGBM:** gradient-boosted trees; strong tabular challengers; more tuning/explainability cost.
- **kNN:** instance-based similarity model; useful diversity, weak when high-dimensional sparse categories dominate.
- **OOF-calibrated stack:** combines base model out-of-fold predictions through a meta model to avoid training-on-self predictions.

Edge Cases / Weak Spots

- **Rare positives:** only 58 positives in temporal test; PR-AUC matters more than accuracy.
- **Label maturity:** last 180 days had 36% label coverage and 32 positives; deployment gate fails for data maturity, not model performance.
- **Recent holdout collapse:** very few positives can make F1 unstable or zero.
- **Distribution drift:** 1994-2021 training behavior may not match 2022 order patterns.
- **Rolling features:** target-dependent rollups inherited label-maturity bias and hurt performance.

Plain-English Label Maturity

A newer order may not have had enough time to reveal whether it truly became a backorder. It is like grading a test before every student has turned it in. The model may be fine, but the answer key is incomplete.

Plain-English Recent Holdout Collapse

The recent test window had very few actual backorders. That is like judging a fire alarm during a week with almost no fires: one missed event or one false alarm can swing the score dramatically.

Stack Choices: Why Each

- **Python / pandas:** reproducible ETL and feature construction.
- **scikit-learn pipelines:** consistent preprocessing, imputation, one-hot encoding, and repeatable evaluation.
- **Temporal validation:** mirrors future deployment better than random splits.
- **PR-AUC + F1:** better for rare-event classification than accuracy.
- **HTML/PDF reports:** make model evidence reviewable for non-code audiences.

Likely Questions

- **Why not use accuracy?** With ~3.4% positives, a dumb all-negative model looks accurate but is useless.
- **Why temporal split?** Random mixing can accidentally let future patterns into the test. Older-to-newer testing asks the real question: can past history predict future backorders?
- **Why LR if stack was selected?** Stack won the rule-selected inner process, but LR was better, simpler, and more deployable on outer temporal holdout.
- **Why did feature engineering hurt?** Rolling features amplified label maturity and train/test drift.
- **What would you do next?** add ATP/inventory/lead-time signals, use narrower rolling windows, and model by plant/material cohort when positives are sufficient.

Practical Use Even If Not Deployed

- Risk-ranked watchlist for planners to review high-risk order lines.
- Data quality dashboard: label coverage, positives by month, drift checks.
- Threshold simulation: choose operating point based on cost of missed backorder vs false alarm.
- Procurement/planning triage: where to investigate inventory buffers, lead-time assumptions, and confirmation gaps.
- Governance artifact: proves what is ready, what is blocked, and what data would move the ceiling.

Future Plans: How I Would Improve This

Short version: the next lift probably comes from better business signals, not just fancier algorithms.

- **Add better order-time signals:** available-to-promise status, inventory buffer, supplier/material lead time, plant capacity, purchase order timing, and customer/material history.
- **Score at more than one moment:** keep the order-time model, then add a second checkpoint closer to ship date when more inventory and confirmation data is available.
- **Use fresher training windows:** train on recent years instead of decades if business patterns have changed.
- **Split the problem into smaller groups:** build separate models by plant, material family, or customer segment when there are enough backorder examples.
- **Change the model structure if needed:** move from one universal classifier to a staged system: first flag possible risk, then estimate urgency, then recommend planner action.
- **Improve the labels first:** wait for outcomes to mature before judging the newest orders, so the model learns from a complete answer key.

Quick Glossary

Accuracy: percent of total predictions correct. *Analogy:* a quiz score, but misleading if almost every answer is "no backorder."

Precision: when the model says "risk," how often it is right. *Analogy:* how many alarms were real fires.

Recall: how many real backorders the model caught. *Analogy:* how many actual fires the alarm detected.

F1: balance between precision and recall. *Analogy:* one score for being careful and not missing too much.

PR-AUC: ranking quality for rare events. *Analogy:* how well the model puts the risky orders near the top of the watchlist.

ROC-AUC: overall ability to rank risky above non-risky orders. *Analogy:* sorting suspicious packages ahead of normal ones.

Temporal Split: train on older orders, test on newer orders. *Analogy:* practice with past games, then predict the next game.

Leakage: accidentally using information that would not be known yet. *Analogy:* studying with some test answers included.

Logistic Regression: simple model that weights each signal. *Analogy:* a checklist where each item adds or subtracts risk.

Random Forest: many decision trees voting together. *Analogy:* asking a panel of judges and taking the majority view.

XGBoost / LightGBM: boosted tree models that learn from prior mistakes. *Analogy:* each new reviewer focuses on what the last reviewer missed.

kNN: predicts from similar past examples. *Analogy:* "orders like this one usually behaved this way."

OOF: out-of-fold predictions from rows a model did not train on. *Analogy:* grading practice answers with a teacher who did not write them.

OOF-Calibrated Stack: combines several models using honest out-of-fold predictions. *Analogy:* a coach trusts each specialist only after seeing how they did on unfamiliar cases.

Threshold: cutoff for calling an order risky. *Analogy:* how sensitive you set a smoke alarm.

Label Maturity: enough time has passed to know the true outcome. *Analogy:* grading only after all tests are turned in.

How To Explain It By Audience

One-sentence opener: I built an early warning system that predicts which SAP order lines are most likely to become backorders, using only information available when the order is placed.

- **Non-technical visitor:** "Think of this like a watchlist. The model does not magically fix inventory, but it helps planners know which orders deserve attention first."
- **Business visitor:** "The tradeoff is missed backorders versus false alarms. A stricter alarm wastes less time; a more sensitive alarm catches more risk."
- **Technical visitor:** "I tested older orders against newer orders, removed future-looking fields, and used rare-event metrics because backorders are uncommon."
- **If someone asks why deployment is blocked:** "The model passed the basic performance checks, but the newest labels are not complete enough yet. The answer key is still being filled in."
- **If someone asks what I would improve:** "I would add richer business signals: inventory buffer, available-to-promise status, lead times, purchase orders, and plant/material-specific patterns."

Question to ask them: Would you rather miss fewer backorders, or avoid sending planners too many false alarms?

If You Get One Hard Question

Answer frame: "The model clears basic performance gates, but I would not call it production-ready until label maturity improves. The capstone value is that it separates model quality from data readiness and gives a concrete roadmap for operational deployment."